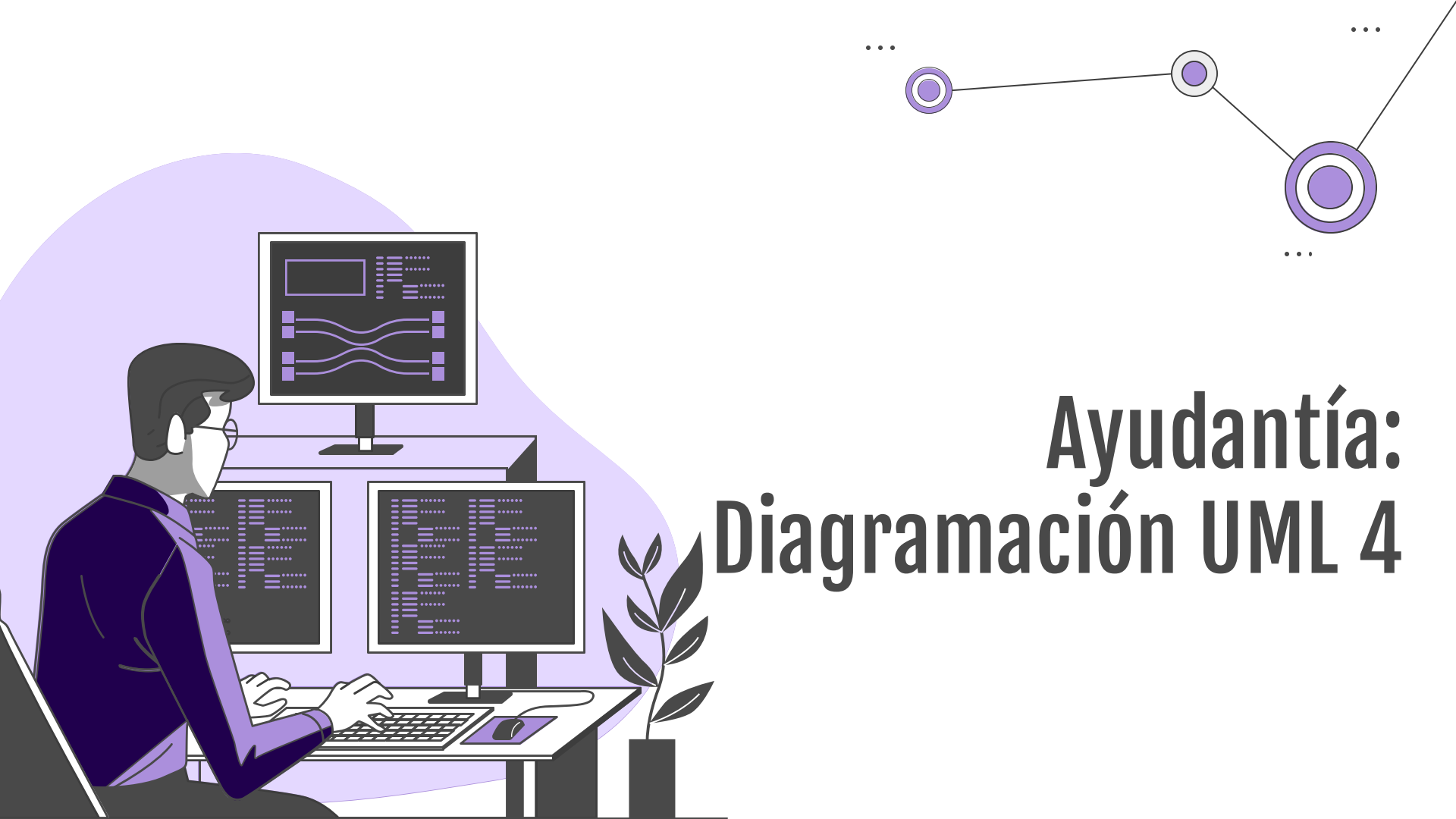
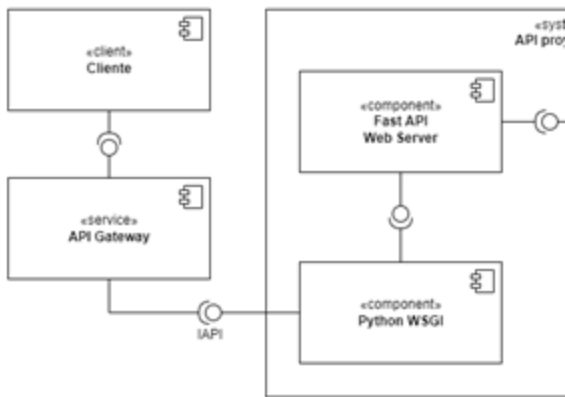




Ayudantía: Diagramación UML 4

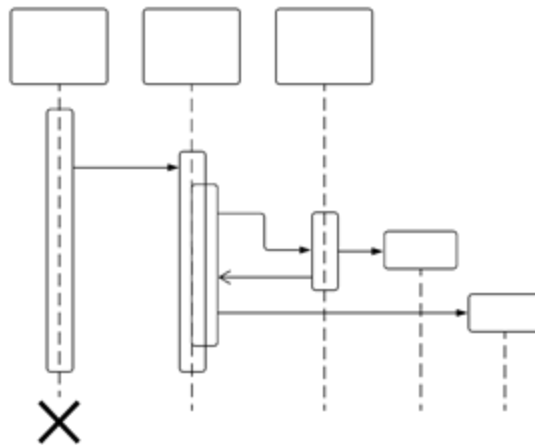
Diagramas importantes para este curso

Component Diagram (estructura)



Conectores (Cómo interactúan)
Componentes (Cajas y sus subcajas)
Interfaces (C requiere y O provee)

Sequence Diagram (comportamiento)



Actores (Columnas)
Mensajes (Señales entre columnas)

Métodos clave



Inductivo: detalle a lo general

Tengo que armar un chat en tiempo real: debe aplicar invocación implícita

Tengo que almacenar fotos: hay que introducir un tipo de storage

...



Deductivo: general al detalle

Debo cubrir 1000 usuarios por hora: un balanceador de carga

Debo tener una arquitectura orientada a servicios (SoA)

...



Referencia

He visto esto en otro lado...

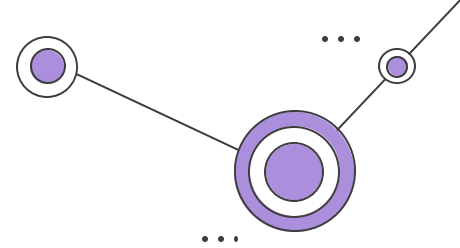
Hay componentes reusables parecidos

...

A decorative network diagram with purple nodes and lines. The nodes are represented by concentric circles, with some having a solid purple center and others being hollow. They are connected by thin black lines. There are three main paths: one in the top right corner, one in the bottom left corner, and one in the bottom center. Each path starts with an ellipsis (...).

**Vamos a la
pizarra**

Interrogación 2025-1



En los últimos años se ha intentado incentivar que las personas puedan ejercitarse correctamente para mantener un estilo de vida más sano y con menos enfermedades provocadas por la vida sedentaria. Es por esto por lo que la empresa LegitSport los contacta para hacer un diseño de un sistema que quieren desarrollar con su equipo.

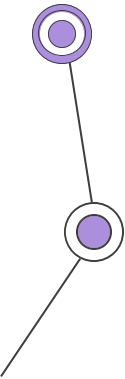
La empresa les indica que quieren realizar un sistema de monitoreo de salud para entrenamientos, acompañado de una aplicación capaz de detectar estos cambios para presentarle ejercicios a la persona de acuerdo con sus capacidades usando Inteligencia Artificial.

De esta forma, les dicen que el monitoreo se realizará usando una polera con microsensors que recolectarán distintas métricas del cuerpo humano de la persona que lo esté utilizando. Estos sensores entregarán distintos tipos de información que deberán enviarse a una aplicación del celular de la persona via bluetooth usando un componente en la polera dedicado a mantener esa conexión y reenviar la información obtenida por los sensores.

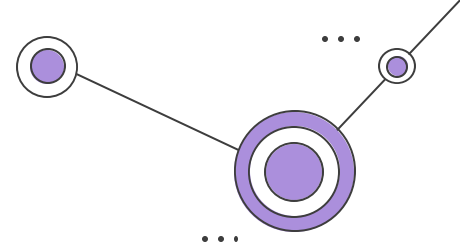
Todos estos datos deberán ser recibidos por la aplicación donde, usando una lista de reglas específicas, deberá tener un componente capaz de usar los datos anteriores y estas reglas para calcular las métricas en detalle como la frecuencia cardíaca, cantidad de oxígeno en sangre, presión en la sangre, etc.

Una vez se tengan los datos, la aplicación debe usar las métricas procesadas para que, mediante el componente ExercisesAI, sea capaz de proponerle los ejercicios siguientes que debería realizar la persona usando las interfaces iMetrics (recibe las métricas) y iExercises (envía los ejercicios).

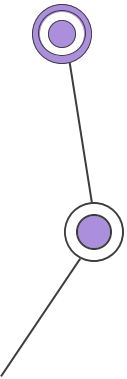
Finalmente, el usuario debe tener la capacidad de conectarse al servidor de LegitSport para iniciar sesión y poder ver los entrenamientos que se le proponen hacer, poder conectar su app a su polera con un click, ver sus métricas procesadas y poder ver un historial de los entrenamientos y datos de salud que hayan sido calculados en la aplicación. Para esto último, al finalizar el día la aplicación encriptará y enviará todos los datos procesados y los entrenamientos propuestos del día almacenados en la memoria local, para luego ser obtenidos cuando se quiera acceder al historial.



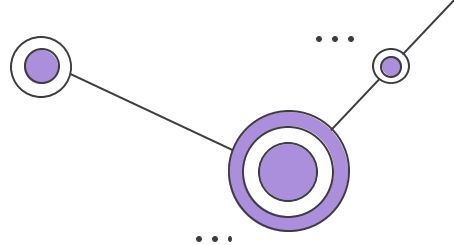
Problema 1



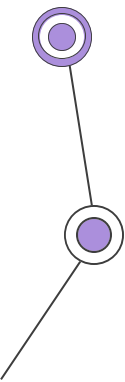
1) Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.



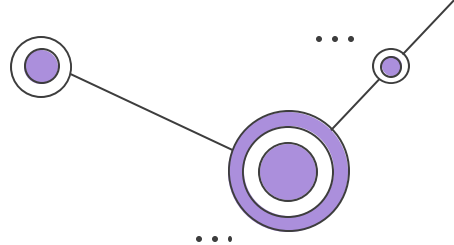
Lo que nos importa



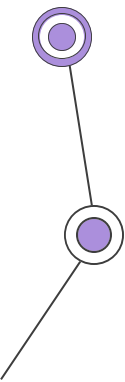
- Que la app sea segura (proteja datos del usuario) - enunciado incluso nos habla de encriptación de los datos
- A nivel de usuario queremos que la carga de datos se haga en un tiempo pertinente
- Queremos que la app sea entendible para el usuario
- Al trabajar con métricas de salud, queremos que las mediciones sean precisas y las recomendaciones sean pertinentes
- Queremos obtener los datos sin necesariamente consumir muchos recursos
- Tenemos los microsensors de la polera, queremos que capten una lista de distintas métricas



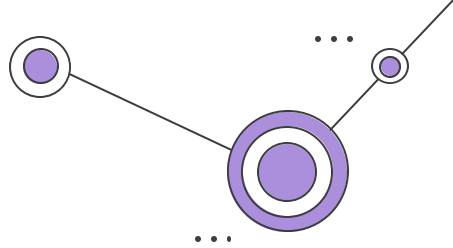
Lo que nos importa



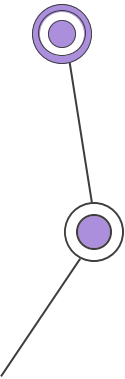
- Que la app sea segura (proteja datos del usuario) - enunciado incluso nos habla de encriptación de los datos
 - o **Seguridad**
- A nivel de usuario queremos que la carga de datos se haga en un tiempo pertinente
 - o **Performance**
- Queremos que la app sea entendible para el usuario
 - o **Usabilidad**
- Al trabajar con métricas de salud, queremos que las mediciones sean precisas y las recomendaciones sean pertinentes
 - o **Confiabilidad**
- Queremos obtener los datos sin necesariamente consumir muchos recursos
 - o **Eficiencia**
- Tenemos los microsensors de la polera, queremos que capten una lista de distintas métricas (funcionalidades)
 - o **Extensibilidad**



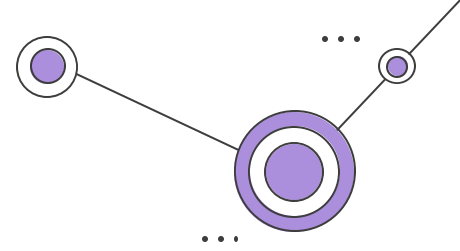
Posibles confusiones



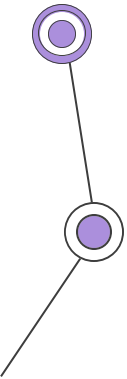
- **Mantenibilidad:** enunciado no hace nunca referencia a algo que nos haga pensar eso
- **Soporte:** En la misma línea, jamás se nos habla de extender el sistema
- **Elasticidad:** No se nos habla nunca de peaks de uso o algo por el estilo
- **Integrabilidad:** Son sistemas propios



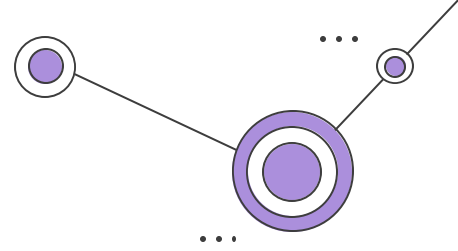
Problema 2



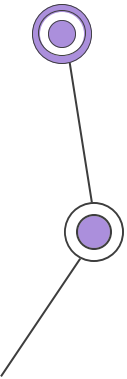
2) Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).



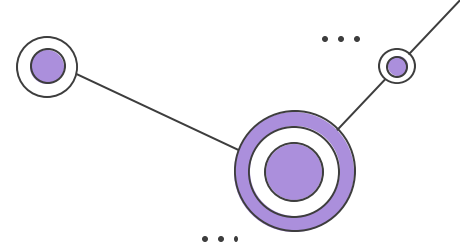
Podemos aplicar directamente



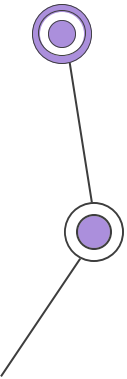
- Obtención de métricas procesadas usando los datos de los sensores y los datos de reglas
- Separación polera, app, servidor
- Uso de broker de eventos para envío de datos
- Uso de MOM (distintas aplicaciones se comuniquen entre sí mediante el envío de mensajes)



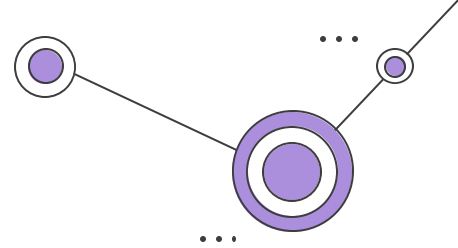
Podemos aplicar directamente



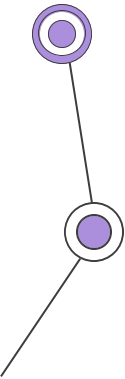
- Obtención de métricas procesadas usando los datos de los sensores y los datos de reglas
 - o Basado en reglas
- Separación por capa, app, servidor
 - o En capas
- Uso de broker de eventos para envío de datos
 - o Basado en eventos
- Uso de MOM (distintas aplicaciones se comunican entre sí mediante el envío de mensajes)
 - o Pub/Sub



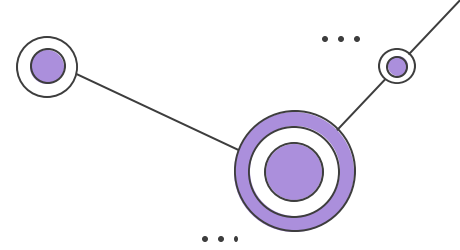
Confusiones / No Aplican



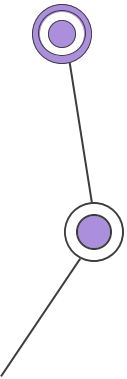
- **Código móvil:** Generalmente usado para IoT, en este caso no aplica ya que no queremos enviar código, solo las métricas
- **Máquina virtual:** No hay necesidad de virtualizar todo el OS, no queremos emular el dispositivo
- **Pizarra:** No se espera un motor de procesos de muchas fuentes de datos.
- **Cliente Servidor:** Hay 3 capas
- **Main y subrutinas:** No hay un punto central gatillante de subrutinas



Problema 3

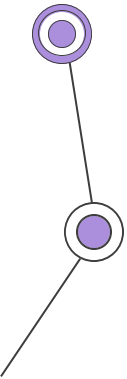
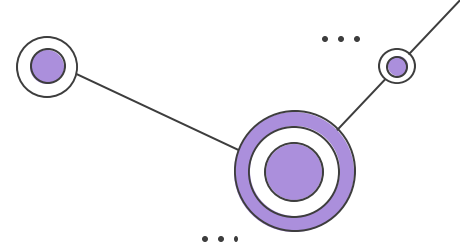


3) Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado.

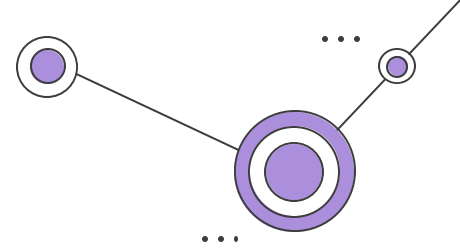


Importante

- Ser consistentes con sus NFRS y Estilos definidos previamente
- Tener componentes conectados
- Seguir estilo de UML visto en clases / ayudantías



Interrogación 2025-1



En los últimos años se ha intentado incentivar que las personas puedan ejercitarse correctamente para mantener un estilo de vida más sano y con menos enfermedades provocadas por la vida sedentaria. Es por esto por lo que la empresa LegitSport los contacta para hacer un diseño de un sistema que quieren desarrollar con su equipo.

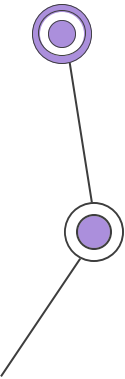
La empresa les indica que quieren realizar un sistema de monitoreo de salud para entrenamientos, acompañado de una aplicación capaz de detectar estos cambios para presentarle ejercicios a la persona de acuerdo con sus capacidades usando Inteligencia Artificial.

De esta forma, les dicen que el monitoreo se realizará usando una polera con microsensors que recolectarán distintas métricas del cuerpo humano de la persona que lo esté utilizando. Estos sensores entregarán distintos tipos de información que deberán enviarse a una aplicación del celular de la persona via bluetooth usando un componente en la polera dedicado a mantener esa conexión y reenviar la información obtenida por los sensores.

Todos estos datos deberán ser recibidos por la aplicación donde, usando una lista de reglas específicas, deberá tener un componente capaz de usar los datos anteriores y estas reglas para calcular las métricas en detalle como la frecuencia cardíaca, cantidad de oxígeno en sangre, presión en la sangre, etc.

Una vez se tengan los datos, la aplicación debe usar las métricas procesadas para que, mediante el componente ExercisesAI, sea capaz de proponerle los ejercicios siguientes que debería realizar la persona usando las interfaces iMetrics (recibe las métricas) y iExercises (envía los ejercicios).

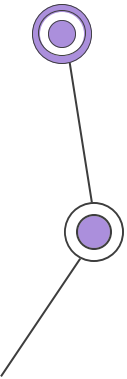
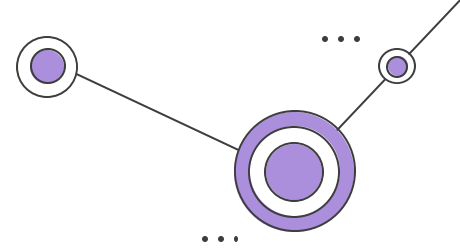
Finalmente, el usuario debe tener la capacidad de conectarse al servidor de LegitSport para iniciar sesión y poder ver los entrenamientos que se le proponen hacer, poder conectar su app a su polera con un click, ver sus métricas procesadas y poder ver un historial de los entrenamientos y datos de salud que hayan sido calculados en la aplicación. Para esto último, al finalizar el día la aplicación encriptará y enviará todos los datos procesados y los entrenamientos propuestos del día almacenados en la memoria local, para luego ser obtenidos cuando se quiera acceder al historial.



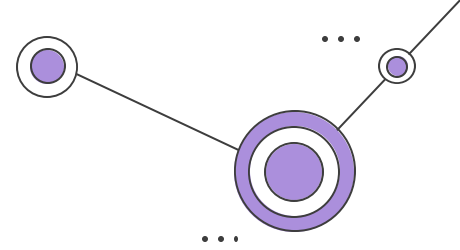
Interrogación 2025-1

Podemos identificar varias cosas:

- **Poleras (IoT)**
 - **microsensores**
 - **MOM**
 - **Componentes Bluetooth**
- **API**
 - **DB Users**
 - **DB Historial**
 - **Endpoint y lógica de ambos + auth**
- **Además: API Gateway y Authorizer externo si aplica**
- **Nos falta lo más complejo: app mobile (siguiente página)**



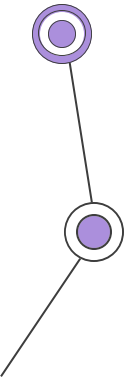
Interrogación 2025-1

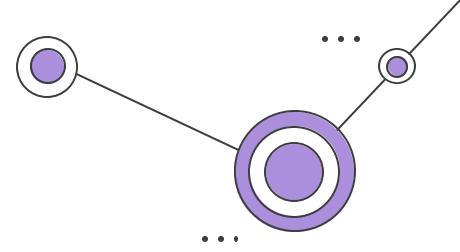


Podemos identificar varias cosas:

App mobile (dentro de teléfono por lo que además debemos diagramarlo)

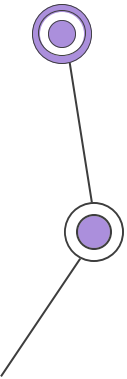
- **Front:**
 - **Views y login + gatillador bluetooth**
- **Exercises (AI)**
 - **ExercisesAI**
 - **Exercises Storer**
- **Procesador de métricas**
 - **DB de rules y local DB (ram)**
 - **Receptor y motor de inferencia**
- **Cifrado**
 - **Encrypt and Decrypt**
 - **Llamada a la API y manejo de tokens**
- **Adicionales**
 - **Conexión al bluetooth desde bt teléfono**
 - **BD local**
 - **Componente para acceder a data, historial, limpiar bd local, etc..**

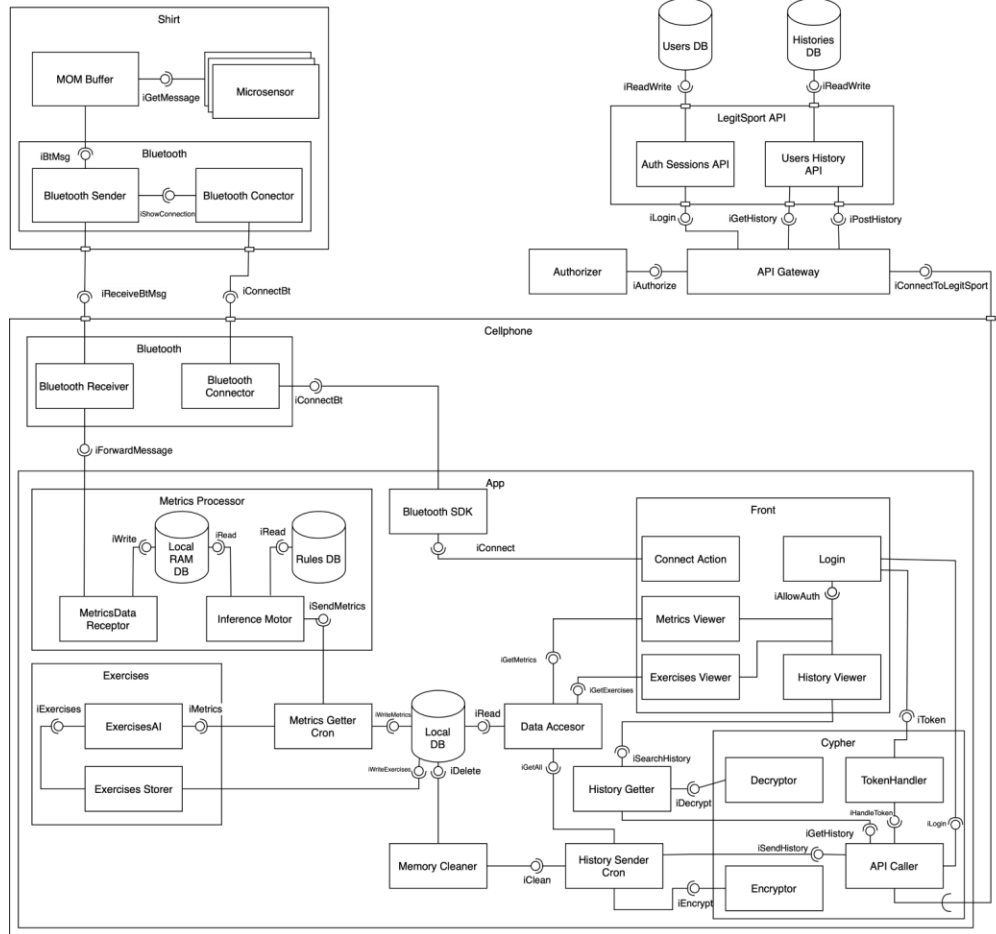


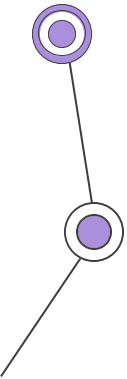
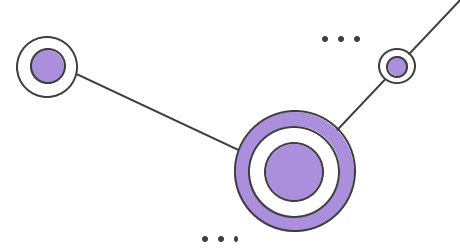
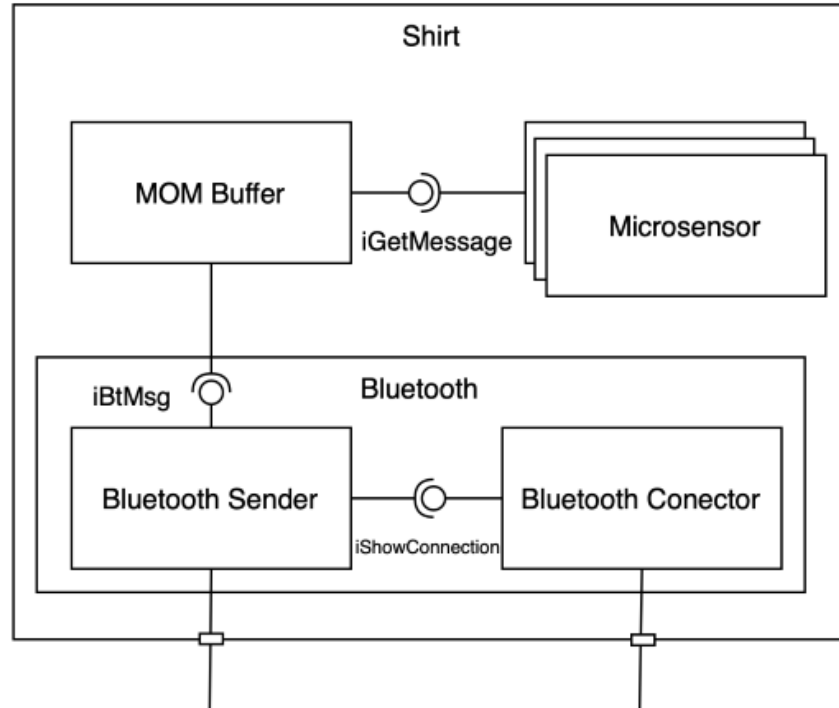


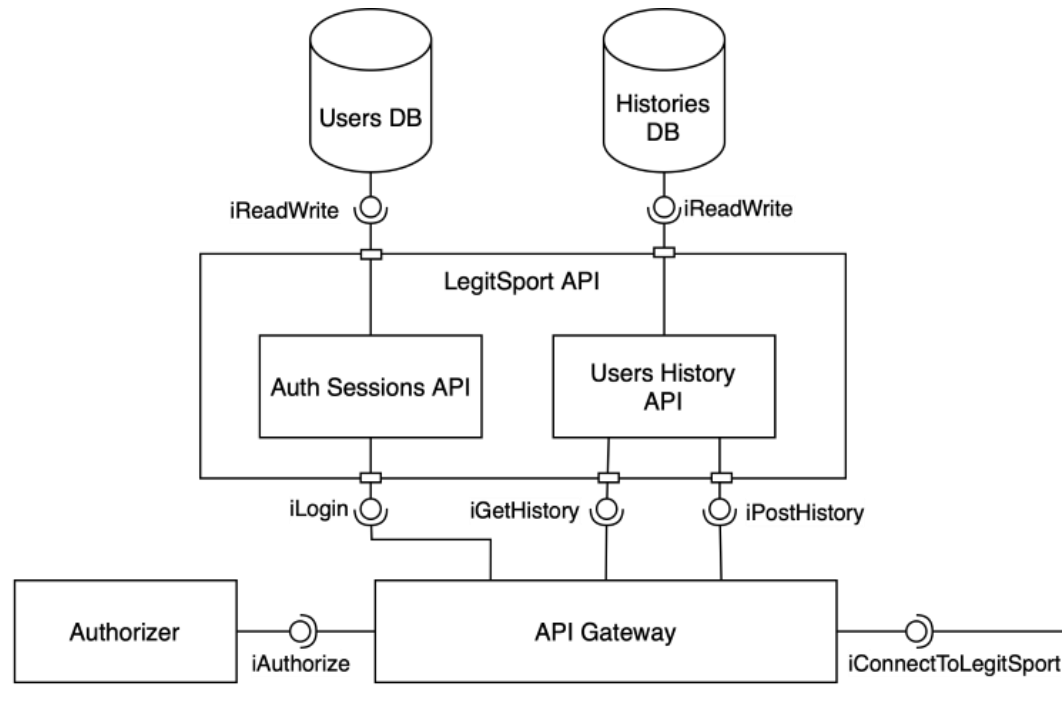
A diagramar!

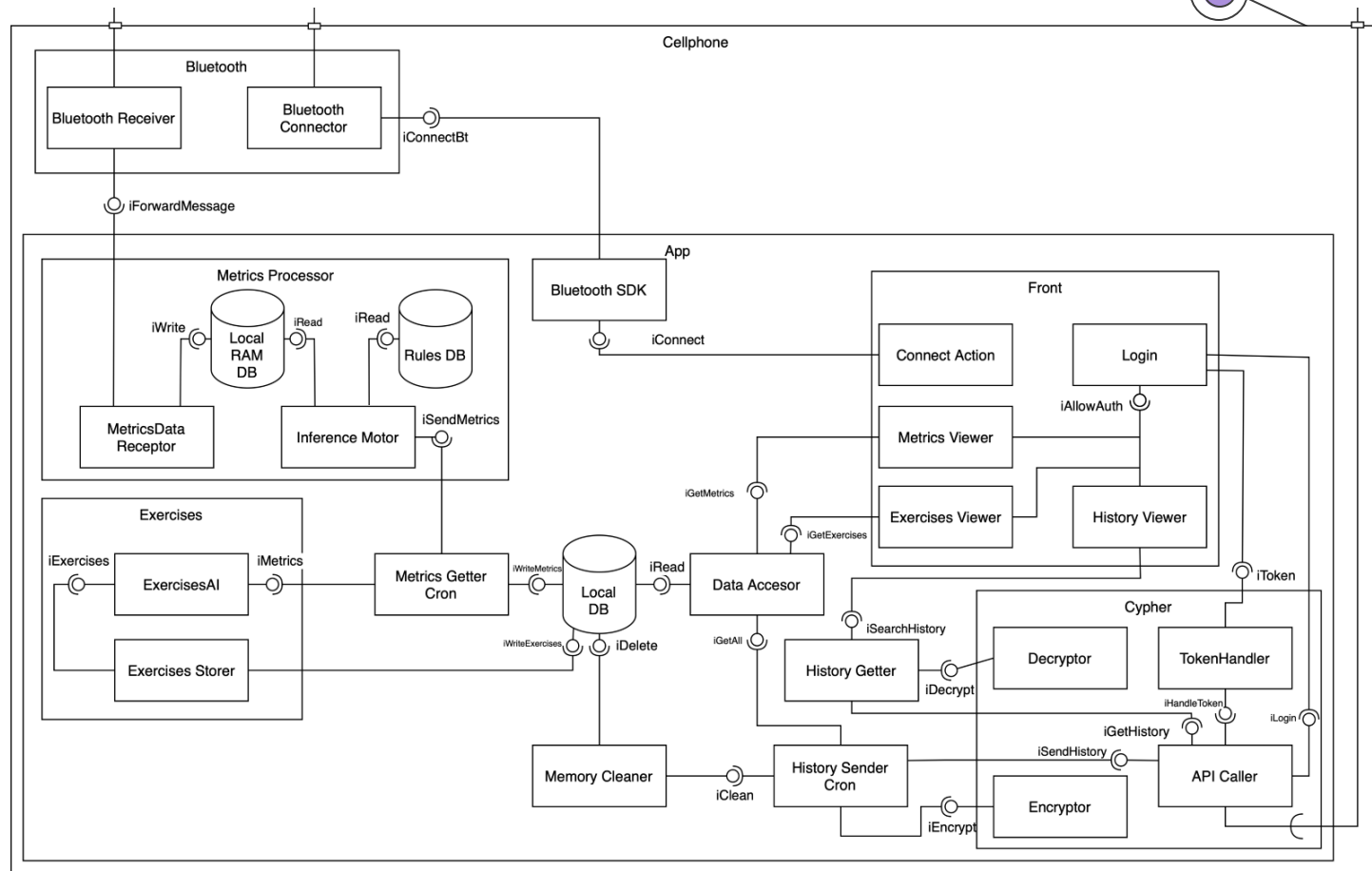
https://drive.google.com/file/d/1X0sGLhbu4Eb4hogjYe7c3ByWlYKd1lwM/view?usp=drive_link



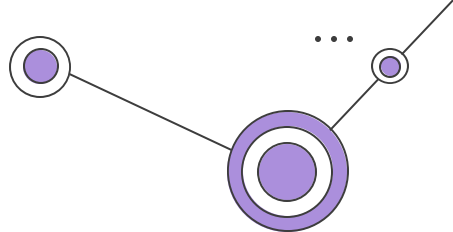








Problema 1



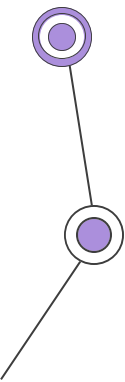
Se les asigna la idea de modelar un sistema de una nueva Red Social que espera muchos usuarios. La idea es que les permita revisar dentro de la web si las contraseñas de sus correos han sido transgredidos o filtradas.

Se les indica que esta red social funciona por foros (tipo Reddit) entre todos los usuarios para que se den indicaciones sobre cómo combatir las vulnerabilidades que han encontrado y conversar de temas X, además de poder buscar un foro por nombre o por tipos de respuestas que se dan.

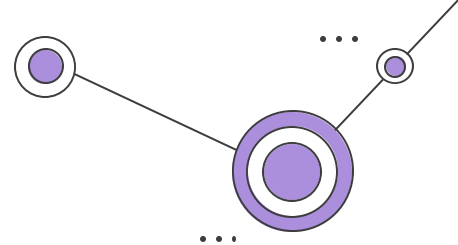
Por otro lado, para la investigación de vulnerabilidades se espera que, accionado un botón, su aplicación sea capaz de:

- Revisar contenidos de distintas direcciones IP almacenadas en una blacklist separada a su DB principal y ver si el correo específico ha sido usado y desde cuando (su servicio mágicamente puede ver eso).
- Almacenar en otra DB, filtrar y analizar la gravedad de las vulnerabilidades encontradas mediante un desglose de estadísticas.
- Mediante una IA realizar un listado de recomendaciones dadas las estadísticas sacadas previamente.

Como la Red Social será usada por muchos usuarios se espera que mantengan un correcto control del escalamiento de esta, recomendándoles usar distintos servicios para estas funcionalidades previamente desglosadas, y que su aplicación principal sea capaz de manejar una gran carga sin perder velocidades de respuesta ni su disponibilidad.

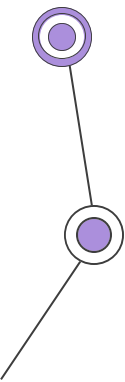


Problema 1

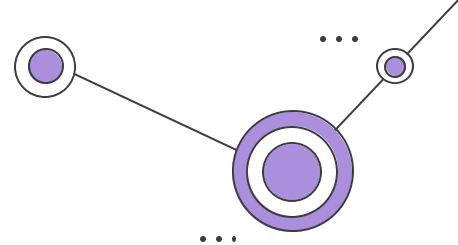


Conteste:

1. ¿Con qué servicio/herramienta podemos afrontar la necesidad de tener muchos usuarios dinámicamente en el contexto global de la aplicación?
2. ¿Qué RNF creen que son necesarios priorizar en este problema? (al menos 3)
3. ¿Qué estilos de arquitectura creen que aplican de mejor manera como solución al problema? (al menos 2)
4. Diseñar el UML con lo decidido

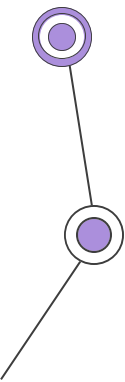


Problema 1

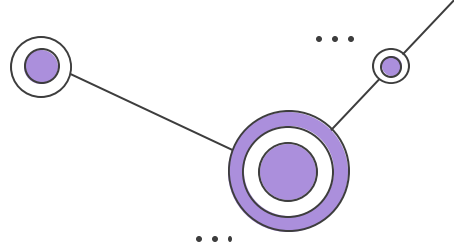


Conteste:

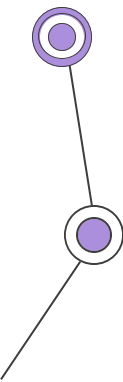
1. ¿Con qué servicio/herramienta podemos afrontar la necesidad de tener muchos usuarios dinámicamente en el contexto global de la aplicación?
2. ¿Qué RNF creen que son necesarios priorizar en este problema? (al menos 3)
3. ¿Qué estilos de arquitectura creen que aplican de mejor manera como solución al problema? (al menos 2)
4. Diseñar el UML con lo decidido



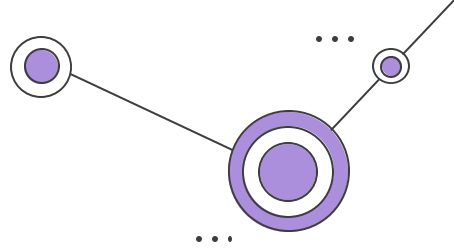
Solución propuesta Problema 1



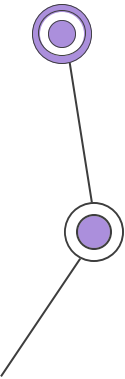
- ¿Con qué servicio/herramienta podemos afrontar la necesidad de tener muchos usuarios dinámicamente en el contexto global de la aplicación?
 - a. Podemos usar un load balancer para la API de mensajes/posts.
 - b. Se puede implementar caché para disminuir la carga del servidor
 - c. Se aceptan otras opciones



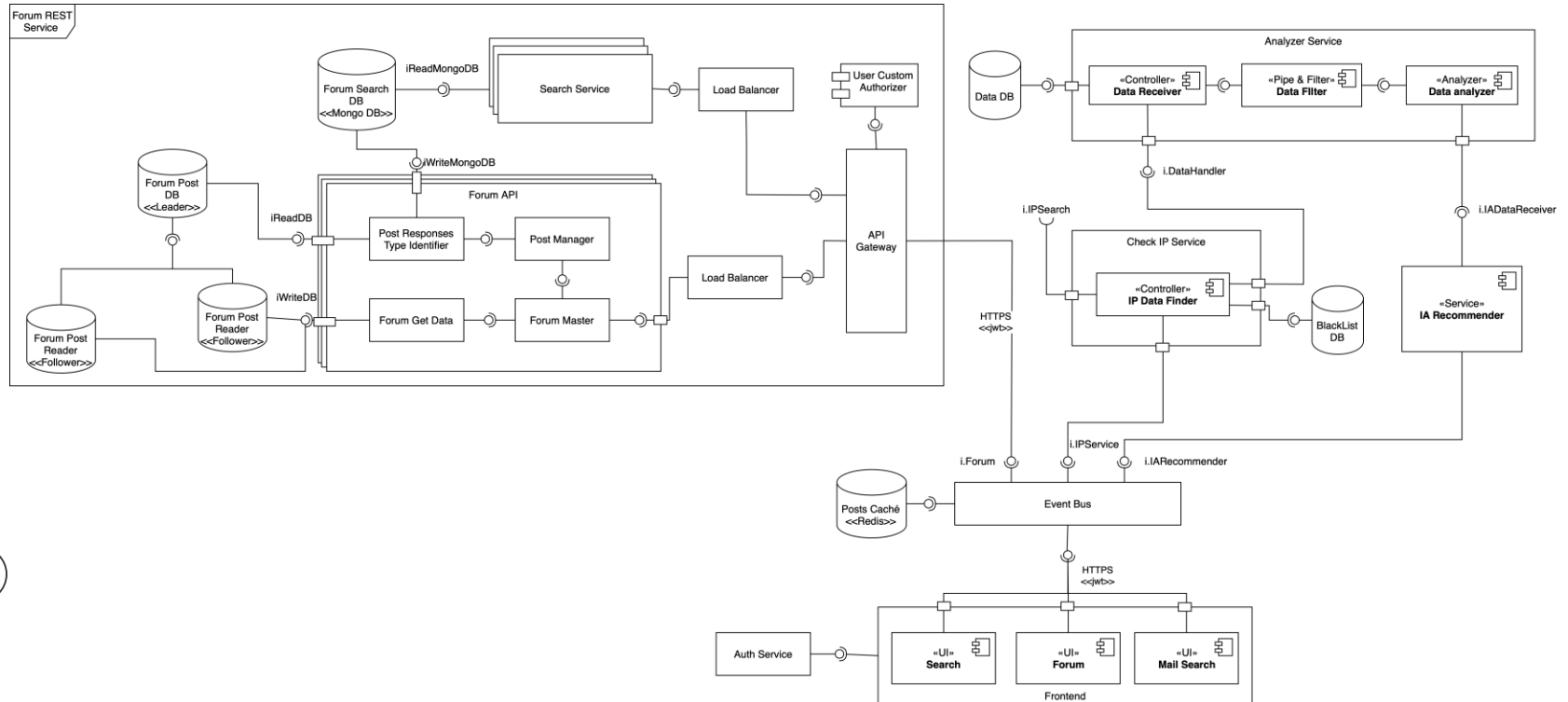
Solución propuesta Problema 1



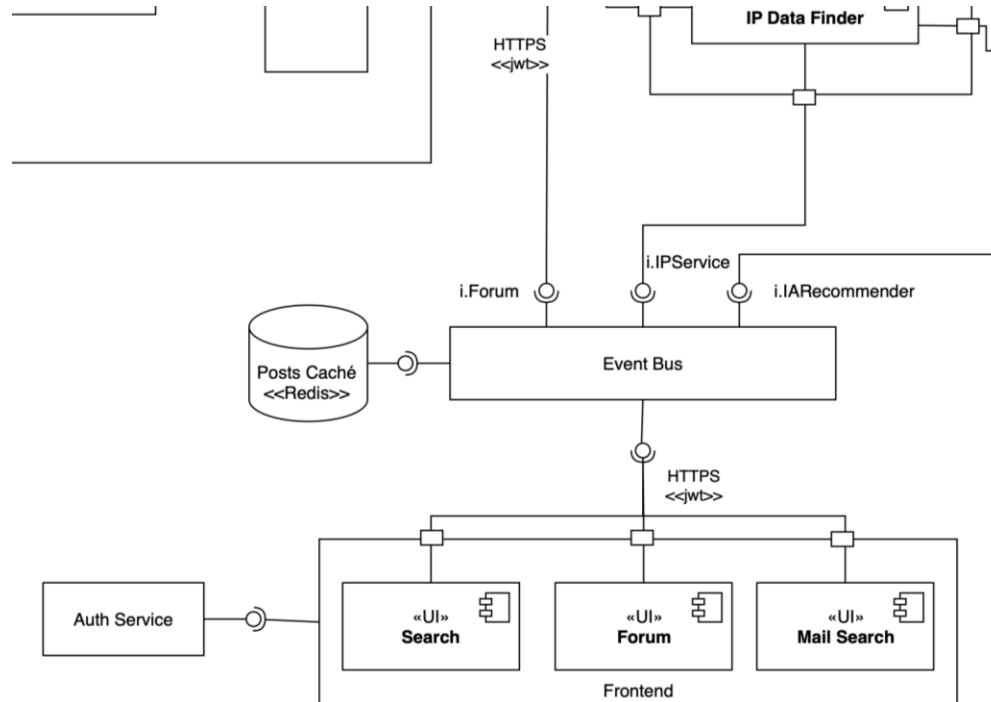
- ¿Qué RNF creen que son necesarios priorizar en este problema? (al menos 2)
 - a. Disponibilidad
 - b. Elasticidad (puede ser escalabilidad)
 - c. Performance
- ¿Qué estilos de arquitectura creen que aplican de mejor manera como solución al problema? (al menos 2)
 - a. EDA (Basado en eventos)
 - b. En capas



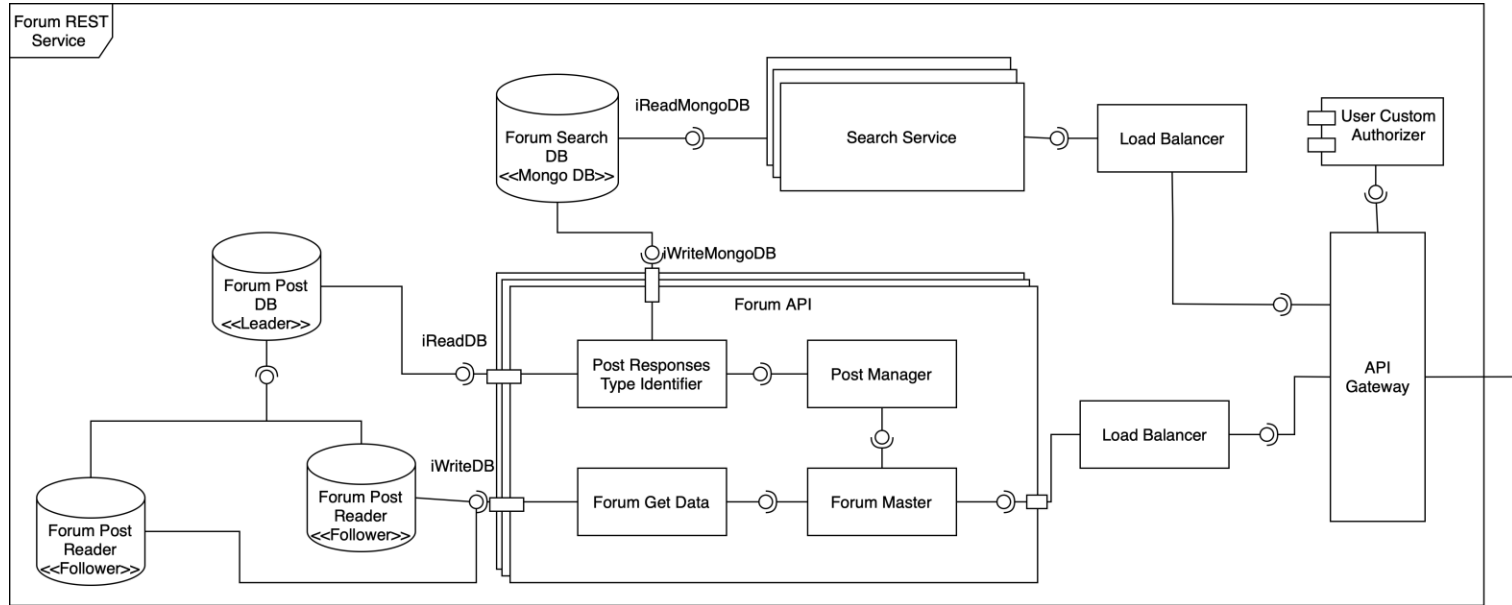
Problema 1 Solución propuesta



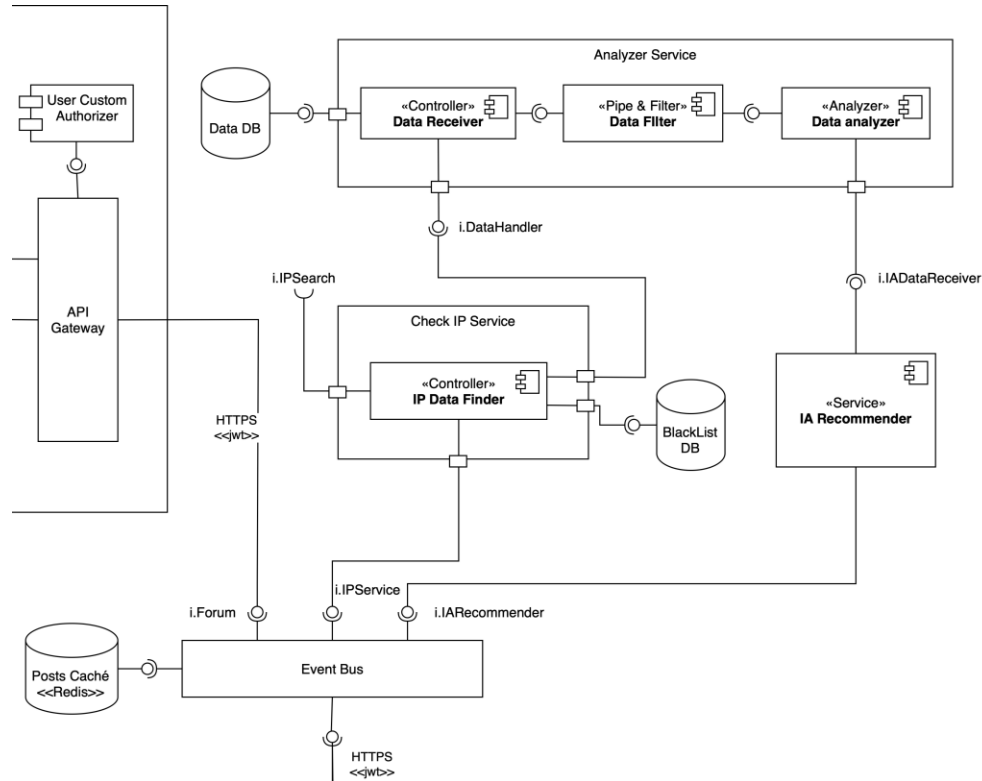
Problema 1 Solución propuesta



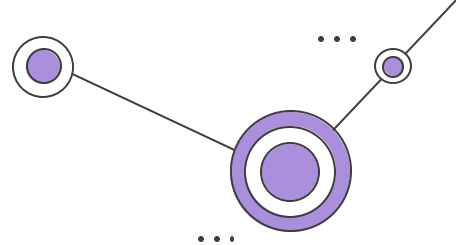
Problema 1 Solución propuesta



Problema 1 Solución propuesta



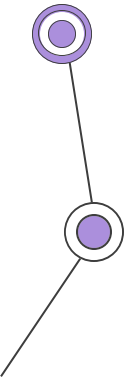
Problema 1

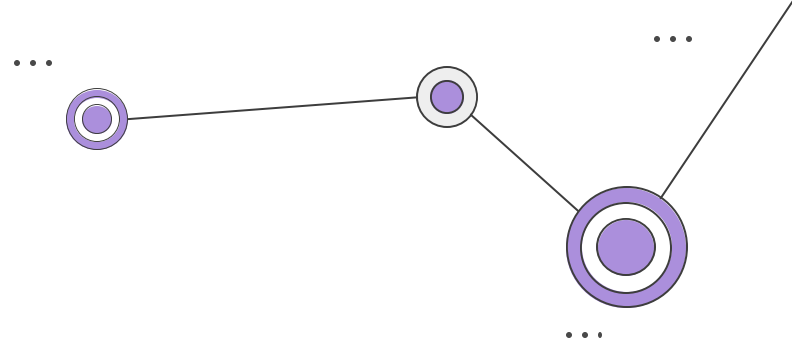
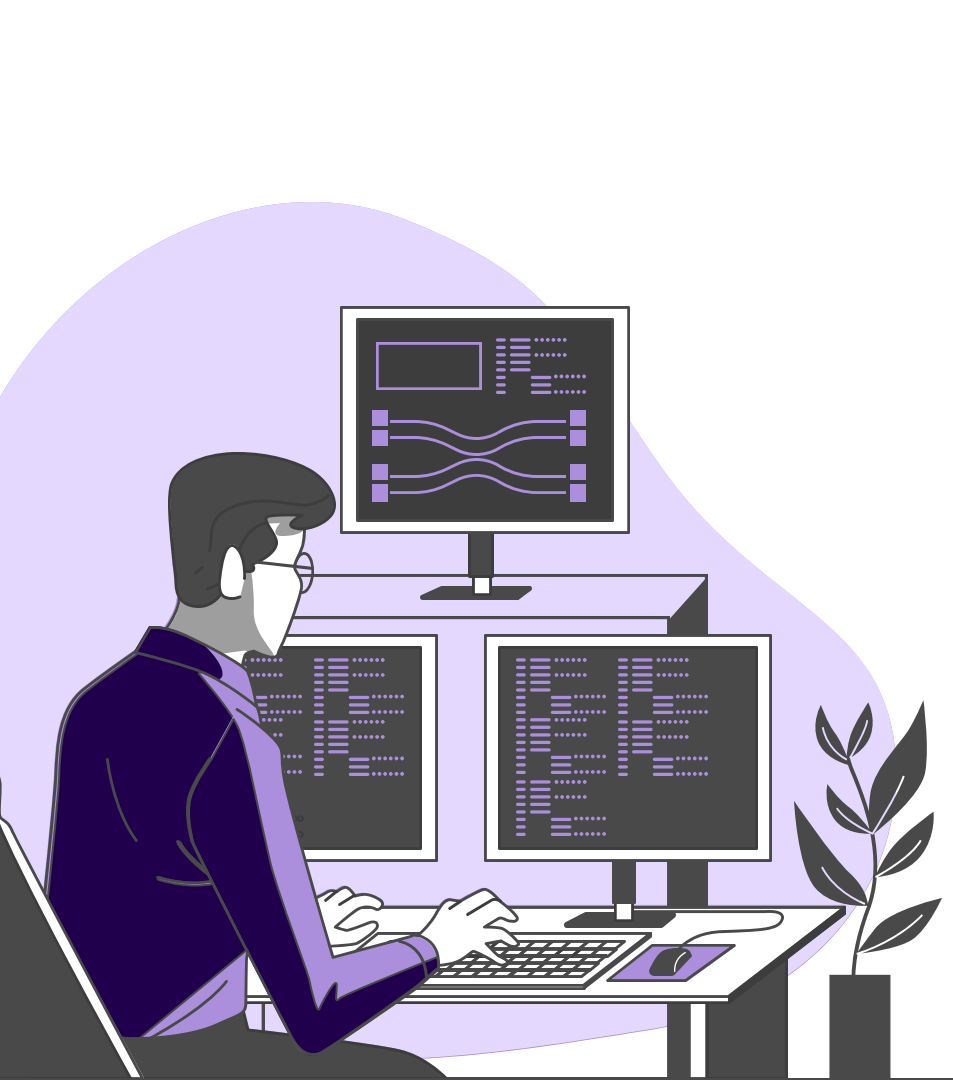


Enunciado extra de desafío:

Junto a esto, se les indica que deben disponibilizar una forma de compartir sus resultados de vulnerabilidades con otras aplicaciones que estén interesadas en combatir estos tipos de ataques y vulneraciones. Debe encontrar una manera segura y eficiente de permitir el envío de esta información.

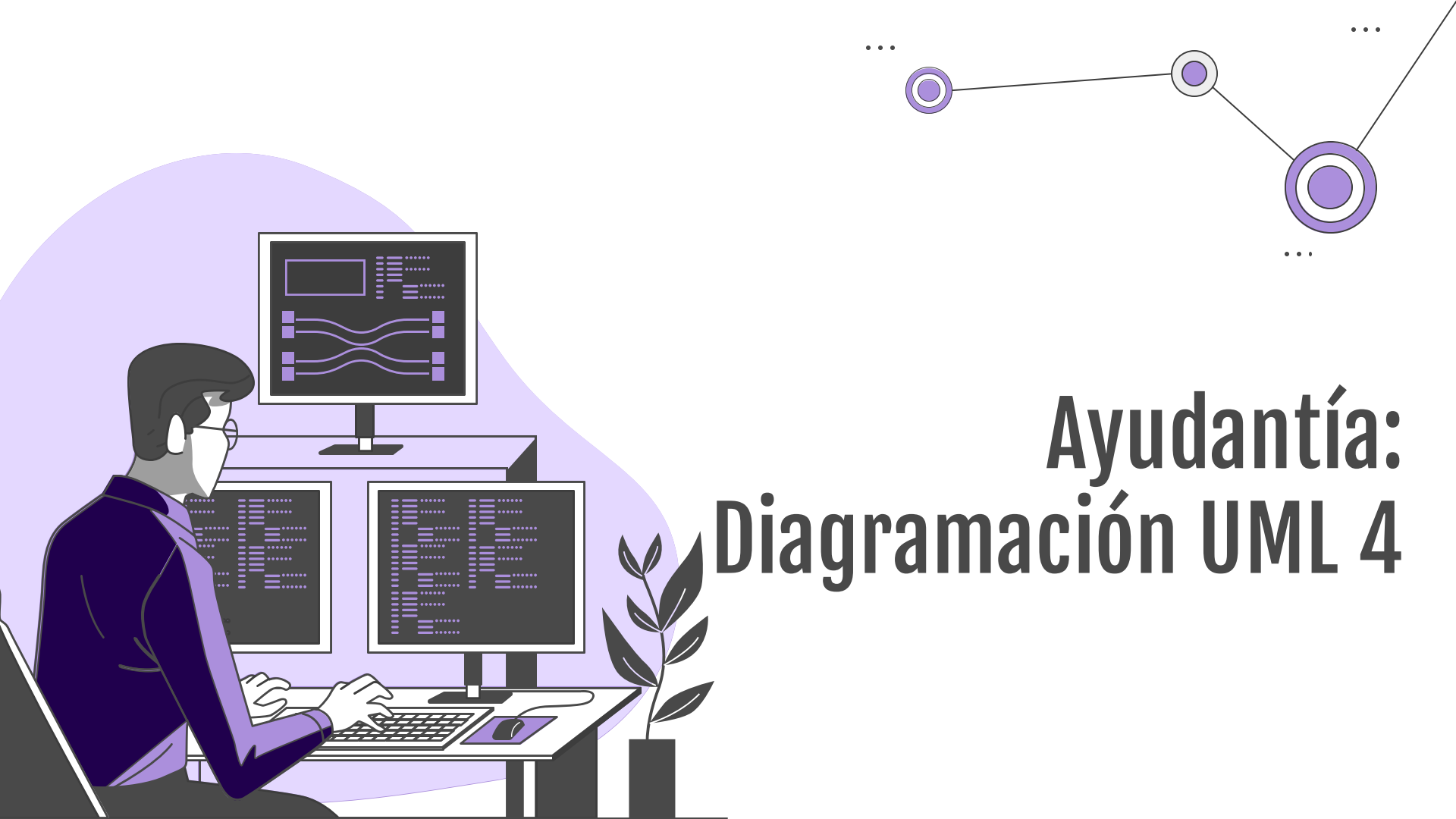
Agregue una solución a su diagrama pensando en las tecnologías que puede usar





Recomendaciones:

1. Lean con calma el enunciado
2. Repasen ayudantías/ Interrogaciones pasadas
3. Apliquen lo que han aprendido en clases/ ayudantías, no busquen online sus respuestas



Ayudantía: Diagramación UML 4